

Template Method Design Pattern

Intent

Define the **skeleton of an algorithm** in a base class, but let subclasses fill in **specific steps** without changing the algorithm's overall structure or order. The "template" is a method that lays out the fixed sequence of steps; some steps are implemented once in the base class, others are left for subclasses to override.

Here the algorithm is **making a hot beverage**: boil water → brew → pour into cup → add condiments. The *order* is fixed for every beverage, but *brewing* and *condiments* differ between coffee and tea.

UML class diagram

```
<<abstract>> BeverageMaker +-----+ | +makeBeverage() {final}
<-- TEMPLATE: fixed order | boilWater() (concrete) | | brew() (abstract) | | pourInCup()
(concrete) | | addCondiments() (abstract) | +-----^-----^-----+ | |
CoffeeMaker TeaMaker drip + sugar/milk steep + lemon
```

The players

```
template/BeverageMaker abstract base: holds the template method + steps
template/concrets/CoffeeMaker fills in brew()/addCondiments() the coffee way
template/concrets/TeaMaker fills in brew()/addCondiments() the tea way
TemplateMethodDesignPattern the demo – makes a coffee, then a tea
```

The code, line by line

BeverageMaker — the abstract base with the template

```
public abstract class BeverageMaker { public final void makeBeverage() { boilWater();
brew(); pourInCup(); addCondiments(); } public abstract void brew(); public abstract void
addCondiments(); void boilWater() { System.out.println("Boiling water"); } void
pourInCup() { System.out.println("Pouring into cup"); } }
```

This one class contains the entire pattern. Four kinds of members, each with a distinct role:

- `public final void makeBeverage()` — **the *template method***. This is the algorithm skeleton. It calls the four steps **in a fixed order**. It is `final` on purpose (see *Why final* below): subclasses can change the individual steps but **cannot** change the recipe's structure or sequence.
- `public abstract void brew(); / public abstract void addCondiments();` — **the *primitive operations***. These are the steps that vary per beverage. They are abstract, so `BeverageMaker` refuses to say how they work and **forces** every subclass to supply its own version.
- `boilWater() / pourInCup()` — **the *concrete (invariant) steps***. These are identical for every beverage, so the base class implements them **once** and shares them with all subclasses. No subclass needs to (or should) reimplement them.

So `makeBeverage()` is a mix: it orchestrates two shared steps and two subclass-supplied steps into one unchangeable sequence.

`CoffeeMaker` — one concrete implementation

```
public class CoffeeMaker extends BeverageMaker { @Override public void brew() {
    System.out.println("Dripping coffee through filter"); } @Override public void
    addCondiments() { System.out.println("Adding sugar and milk"); } }
```

- Extends `BeverageMaker` and supplies **only** the two varying steps. It says nothing about boiling water, pouring, or the *order* — all of that is inherited from the template.
- Notice there is **no `makeBeverage()` here**. The subclass provides *ingredients for steps*, not the algorithm. That inversion is the whole point.

`TeaMaker` — the other implementation

```
public class TeaMaker extends BeverageMaker { @Override public void brew() {
    System.out.println("Steeping the tea"); } @Override public void addCondiments() {
    System.out.println("Adding lemon"); } }
```

- Same deal, different fillings: steep instead of drip, lemon instead of sugar-and-milk. The skeleton it runs is byte-for-byte the same one `CoffeeMaker` runs.

`TemplateMethodDesignPattern` — the demo

```
BeverageMaker coffeeMaker = new CoffeeMaker(); coffeeMaker.makeBeverage(); BeverageMaker
teaMaker = new TeaMaker(); teaMaker.makeBeverage();
```

- Both are referenced through the **base type** `BeverageMaker`. The caller invokes the **same** method, `makeBeverage()`, on both.

- What differs at runtime is decided by **polymorphism**: when `makeBeverage()` reaches `brew()`, Java dispatches to the actual object's override (`CoffeeMaker.brew()` OR `TeaMaker.brew()`). The caller never calls `brew()` itself — the template calls it for you.
-

Why the design decisions

Why put the algorithm in the base class instead of each subclass?

Because the **order and structure are the part that must not vary**. Every beverage is made by the same four-step recipe in the same sequence. If each subclass wrote its own `makeBeverage()`, the shared sequence would be **duplicated** across all of them, and any subclass could accidentally get the order wrong (pour before brewing) or forget a step. Centralizing the skeleton in one place means the recipe is defined **once** and reused. This is the pattern's core benefit: *code reuse of the algorithm's structure*.

Why is the template method `final`?

`final` prevents subclasses from overriding `makeBeverage()`. That's deliberate: the base class is saying **"you may customize the steps, but you may not rewrite the recipe."** Without `final`, a subclass could override the template and change the order, skip `boilWater()`, or otherwise break the invariant that the pattern is meant to protect. Making it `final` locks in the algorithm's shape while still allowing per-step variation through the abstract methods.

Why are `brew()` and `addCondiments()` abstract?

Because they are the steps that genuinely differ and for which the base class has **no sensible default**. Declaring them `abstract`:

1. **Forces** every subclass to provide them — you literally cannot compile a concrete `BeverageMaker` that forgets to define `brew()`. The compiler enforces the contract.
2. Documents exactly which steps are the customization points.

Why are `boilWater()` and `pourInCup()` concrete (and not abstract)?

Because they are **invariant** — the same for every beverage. Implementing them in the base class shares the code and signals "don't touch these." (They have package-private visibility here, which is enough for the template method in the same class to call them; they aren't part of the public customization surface.)

Why is this called "inversion of control" / "the Hollywood Principle"?

Note who calls whom. The subclass does **not** call the base class to run the algorithm; instead the base class's template method calls **down** into the subclass's overridden steps at the right

moments — *"Don't call us, we'll call you."* The high-level flow stays in the base class; the low-level details are supplied by subclasses and invoked *by* the framework method. This is the opposite of the usual "my code calls a library" direction, and it's exactly what lets the base class guarantee the order.

Template Method vs. Strategy (they solve the same varying-behavior problem differently)

- **Template Method (this module)** varies steps via **inheritance** — subclasses override abstract methods, and the algorithm skeleton is fixed at compile time in the base class.
- **Strategy** varies the whole algorithm via **composition** — you inject a different strategy object at runtime.

Template Method is the right choice when the *overall sequence* is fixed and only *specific steps* change; Strategy is better when the *entire* algorithm should be swappable and you want to change it at runtime.

Execution flow (the demo)

```
main ■ ■■■ new CoffeeMaker() (as BeverageMaker) ■ ■■■ makeBeverage() ← the template
method (defined once, in the base) ■ ■■■ boilWater() → "Boiling water" (base class) ■
■■■ brew() → "Dripping coffee through filter" (CoffeeMaker override) ■ ■■■ pourInCup()
→ "Pouring into cup" (base class) ■ ■■■ addCondiments() → "Adding sugar and milk"
(CoffeeMaker override) ■ ■■■ new TeaMaker() (as BeverageMaker) ■■■ makeBeverage() ← the
SAME template method ■■■ boilWater() → "Boiling water" (base class) ■■■ brew() →
"Steeping the tea" (TeaMaker override) ■■■ pourInCup() → "Pouring into cup" (base class)
■■■ addCondiments() → "Adding lemon" (TeaMaker override)
```

Console output:

```
Template Method Design Pattern -----Its time to make coffee. Boiling water Dripping coffee
through filter Pouring into cup Adding sugar and milk -----Now its time to make tea.
Boiling water Steeping the tea Pouring into cup Adding lemon
```

Notice: the **first and third lines of each beverage are identical** (the shared steps), while the **second and fourth differ** (the overridden steps) — and the **order is always the same**. That is Template Method working exactly as intended.

Notes / possible extensions (not changed in the code)

- **Hook methods.** A common Template Method addition is a *hook*: a non-abstract method with an empty or default body that subclasses *may* override to optionally influence the flow (e.g. `boolean customerWantsCondiments()` guarding the `addCondiments()` call). This module keeps it to the two required steps for clarity.
- **Plain main, no Spring.** Like the other pure-behavioral demos here, this one runs from a plain `main` — the pattern is about class structure and needs no container.

Relationship to the other Behavioral patterns

- **Template Method (this module)** — fixed algorithm skeleton, subclasses fill in steps (inheritance).
- **Strategy** — swap the whole algorithm at runtime (composition).
- **Chain of Responsibility** — pass a request along handlers until one handles it.

Reach for Template Method when several classes share the **same overall procedure** but differ in a **few specific steps**, and you want to guarantee the procedure's order while eliminating duplicated structure.